

Experiment- 8

Branch: MCA (AI&ML)	Semester: 2
Student Name: Ajay Rakwal	UID: 25MCI10329
Subject Name: Technical Training	Subject Code: 25CAP-652
Section/Group: MAM-1(A)	Date of Performance: 31-03-2026

Aim of the program:

To apply the concept of Stored Procedures in database operations in order to perform tasks like insertion, updating, deletion, and retrieval of data efficiently, securely, and in a reusable manner within the database system.

Software Requirements

- **Operating System:** Windows / Linux
- **Database Management System:** MySQL / Oracle / PostgreSQL
- **SQL Interface:** MySQL Workbench / Web Based / pgAdmin

Objective

Apply stored procedure concepts for database operations

Procedure of the Practical

- Open **PostgreSQL (pgAdmin)** and connect to the required database.
- Create the **Employee** table with attributes such as empId, name, and salary.
- Create a **stored procedure** to insert employee records into the table.

- Create a **stored procedure** to update employee salary based on empId.
- Create a **stored procedure** to delete employee records from the table.
- Create a **function** to retrieve and display all employee records.
- Execute the procedures (insert, update, delete) and call the function to verify results.
- Observe the changes in the table after each operation.

Practical / Experiment Steps

```
CREATE TABLE Employee (
```

```
    empId INT PRIMARY KEY,
```

```
    name VARCHAR(50),
```

```
    salary INT
```

```
);
```

```
--- INSERT PROCEDURE
```

```
CREATE OR REPLACE PROCEDURE insert_employee(
```

```
    eid INT,
```

```
    ename VARCHAR,
```

```
    esalary INT )
```

```
LANGUAGE plpgsql
```

```
AS $$
```

```
BEGIN
```

```
    INSERT INTO Employee VALUES (eid, ename, esalary);
```

```
END;
```

```
$$;
```

--- UPDATE PROCEDURE

CREATE OR REPLACE PROCEDURE update_salary(

 eid INT,

 new_salary INT)

LANGUAGE plpgsql

AS \$\$

BEGIN

 UPDATE Employee

 SET salary = new_salary

 WHERE empId = eid;

END;

\$\$;

--- DELETE PROCEDURE

CREATE OR REPLACE PROCEDURE delete_employee(

 eid INT)

LANGUAGE plpgsql

AS \$\$

BEGIN

 DELETE FROM Employee

 WHERE empId = eid;

END;

\$\$;

--- RETRIVE DATA USING FUNCTION

CREATE OR REPLACE FUNCTION get_employees()

RETURNS TABLE(empId INT, name VARCHAR, salary INT)

LANGUAGE plpgsql

AS \$\$

BEGIN

 RETURN QUERY

 SELECT * FROM Employee;

END;

\$\$;

--- EXECUTION PART

CALL insert_employee(1, 'Ajay', 10000);

CALL update_salary(1, 60000);

CALL insert_employee(2, 'Jatin', 50000);

SELECT * FROM get_employees();

CALL delete_employee(1);

I/O Analysis (Input / Output)

Input

- Table creation query for Employee
- Stored procedure definitions (INSERT, UPDATE, DELETE)
- Function definition for data retrieval
- Procedure calls for inserting, updating, and deleting records

- SELECT query using the function

Output

- Employee table created successfully
- Records inserted, updated, and deleted correctly using procedures
- Employee data retrieved successfully using the function
- Changes reflected accurately after each operation
- Proper execution of procedural SQL logic in PostgreSQL

OUTPUT:

Table Created:

```
Data Output Messages Notifications
CREATE TABLE

Query returned successfully in 70 msec.
```

DELETE, UPDATE, INSERT PROCEDURE:

```
Data Output Messages Notifications
CREATE PROCEDURE

Query returned successfully in 100 msec.
```

RETRIVE DATA USING FUNCTION

```
Data Output Messages Notifications
CREATE FUNCTION

Query returned successfully in 53 msec.
```

-Insert Procedure

Data Output	Messages	Notifications
≡+	📄	SQL
	empid integer	name character varying
1	1	Ajay
	salary integer	10000

--Update Procedure

Data Output	Messages	Notifications
≡+	📄	SQL
	empid integer	name character varying
1	1	Ajay
	salary integer	60000

-Final before Delete Procedure

Data Output	Messages	Notifications
≡+	📄	SQL
	empid integer	name character varying
1	1	Ajay
2	2	Jatin
	salary integer	50000

--Delete Procedure

Data Output	Messages	Notifications
≡+	📄	SQL
	empid integer	name character varying
1	2	Jatin
	salary integer	50000

Learning Outcome:

- Understand the concept and use of stored procedures and functions in PostgreSQL.
- Learn how to perform CRUD operations (Create, Read, Update, Delete) using procedural SQL.
- Gain practical experience in writing PL/pgSQL blocks.
- Understand how functions can be used to retrieve data efficiently.
- Develop the ability to design modular and reusable database programs for real-world applications.